

Over-the-Air Carrier Phase Synchronization for Distributed Arrays

A lecture: foundations, the reference paper, and this project

Chandler Stevenson

Reference paper [RN23]:

M. Rashid and J. A. Nanzer, "Frequency and Phase Synchronization in Distributed Antenna Arrays Based on Consensus Averaging and Kalman Filtering," *IEEE Trans. Wireless Commun.*, vol. 22, no. 4, pp. 2789–2802, Apr. 2023.
DOI 10.1109/TWC.2022.3213788

July 29, 2026

The problem in one paragraph

Two radio stations, A and B, each generate a 915 MHz carrier from their own crystal oscillator. We want their two carriers to hold a fixed, known phase relationship — within a few degrees — so that their signals add constructively at a distant point, or so that what the two stations *receive* can be combined coherently.

- The two crystals disagree, and their disagreement drifts randomly.
- The only connection between the stations is radio itself.
- So station B must continuously measure its offset from A *over the air* and steer its own electronics to cancel it.

Everything in this lecture is about how well that can be done, what fundamentally limits it, and how the reference paper and this project each attack it.

Roadmap and attribution

- ① **Foundations** (21 slides) — built up slowly; classical sources named per slide
- ② **The reference paper** [RN23] (14 slides) — its model, algorithms, derivation, and results taught with its own equation/figure numbers
- ③ **This project** [proj] (19 slides) — tools and method first, then every number names the command that regenerates it

Tags

[RN23, Eq. 27] = the paper, that equation. [proj: --model compare] = measured here, by that command. Named citations = classical results.

Source: Companion with full derivations: `teaching_document.pdf` (same numbers, same order).

1.0 Vocabulary for the whole lecture

- Carrier / phase** A radio transmits a pure high-frequency sine wave (here 915 million cycles per second) and imprints information on it. *Phase* is where in its cycle the wave is right now — an angle, one full cycle = 2π radians = 360° .
- Units** 1 rad = 57.3° ; 1 mrad = 0.057° (so “28 mrad” means “ 1.6° ”). Decibels compare powers on a log scale: 20 dB = $100\times$; “20 dB SNR” means the signal is $100\times$ stronger than the noise.
- Baseband / IQ samples** The receiver strips off the fast carrier and keeps, at each sampling instant, one *complex number* whose angle is the received phase and whose magnitude is the strength. All digital processing operates on these “IQ samples.”
- Channel** Everything between the antennas: the travel delay plus reflections off objects (*multipath*). At one frequency its net effect is a single complex multiplier — a gain and a phase shift ϕ_c . **AWGN**: the electronic hiss every receiver adds, modeled as Gaussian noise.
- Pilot / correlation** A *pilot* is a pre-agreed known waveform used for measurement. *Correlating* means sliding the known pattern along the received samples, multiplying and summing: the result peaks when aligned, and the peak’s *angle* is the received phase.

1.1 Two waves adding: where the requirement comes from

A wave of one fixed frequency is completely described by an arrow (a *phasor*): length = amplitude, angle = phase; written as a complex number the arrow is $e^{j\phi}$. Two waves arriving at the same point add as arrows, so two equal-strength carriers give a sum that depends only on the angle between them:

$$|1 + e^{j\Delta\phi}|^2 = (1 + \cos \Delta\phi)^2 + \sin^2 \Delta\phi = 2 + 2 \cos \Delta\phi = 4 \cos^2\left(\frac{\Delta\phi}{2}\right)$$

- $\Delta\phi = 0$: the phasors line up; power is $4\times$ one transmitter. Two transmitters, four times the power — the extra factor is the point of coherence. With N elements: N^2 .
- $\Delta\phi = \pi$: the phasors oppose; the sum is *zero*. Being wrong by half a carrier cycle is worse than transmitting alone.

We will use $G = \cos^2(\Delta\phi/2)$, the fraction of ideal two-element gain, as the scorecard for every scheme in this lecture.

1.2 How much phase error is tolerable?

Let each element's phase error ϕ_n be random, zero-mean, Gaussian with standard deviation σ . The mean of a unit phasor with jittered angle is shorter than 1:

$$\mathbb{E}[e^{j\phi_n}] = e^{-\sigma^2/2} \implies \text{mean coherent power} \approx N^2 e^{-\sigma^2}.$$

Why the average arrow is shorter: wobbling the angle moves the arrow mostly *sideways*, and sideways excursions average to zero. But the forward component is $\cos \phi_n$, which is ≤ 1 for every wobble, so its average only loses:

$\mathbb{E}[\cos \phi_n] \approx 1 - \sigma^2/2$ for small errors — which is $e^{-\sigma^2/2}$. Each element still radiates full power; the *resultant* shrinks by that factor per element, hence $e^{-\sigma^2}$ on power.

The working threshold

$\sigma = 18^\circ = 0.314 \text{ rad} \Rightarrow e^{-\sigma^2} = 0.906$: with 18° RMS phase error an array keeps $\geq 90\%$ of ideal gain. This is the standard bar in the distributed-array literature.

For reference: $18^\circ = 314 \text{ mrad}$. The loops later in this lecture reach 23–82 mrad — comfortably inside.

Source: Threshold: Nanzer et al., IEEE TMTT 65(5), 2017 — [RN23]'s reference [4], used throughout it.

1.3 Why this is hard: the scale of a carrier cycle

- At 915 MHz one carrier period is 1.09 ns. A timing disagreement of **1 ns** between stations is 0.915 of a cycle = 329° of phase — more than a full tolerance budget, from one nanosecond.
- Crystals are specified in parts per million. A **1 ppm** frequency error at 915 MHz is 915 Hz: the relative phase sweeps through a *full cycle every 1.09 ms*.
- So “synchronize once” is meaningless. The relative phase must be measured and corrected *continuously*, and the interesting question becomes: what is the steady-state error of that loop?

Also note what coherent *reception* needs: when two stations receive the same signal from a passive source, the source's phase is common to both and cancels when combining. What does not cancel is the phase between the two stations' local oscillators. Station-to-station sync is the whole requirement.

Source: Arithmetic from the carrier frequency; sensing observation: standard.

2.1 Modeling an oscillator

An oscillator's deviation from its nominal frequency is described by two numbers that evolve in time: a phase offset ϕ (rad) and a frequency offset ω (rad/s). Between correction epochs spaced T apart, phase integrates frequency, and both get kicked by noise:

$$\begin{bmatrix} \phi_{k+1} \\ \omega_{k+1} \end{bmatrix} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \phi_k \\ \omega_k \end{bmatrix} + \mathbf{w}_k.$$

- The matrix says: new phase = old phase + $T \times$ frequency; frequency persists. This deterministic part is easy — a controller can predict and cancel it.
- Everything difficult lives in $\mathbf{w}_k$: the random component of real oscillator behavior. The next three slides unpack it.

Source: Standard clock model; used identically in [RN23] Eq. 28 and in this project.

2.2 The dominant noise: white FM = a random walk in phase

“White FM” means the oscillator’s *frequency* jitters independently from sample to sample. Phase is the integral of frequency, so phase takes a small independent random step each sample — a random walk. If each step has standard deviation σ_{pn} at sample rate f_s :

$$\text{var}(\phi(t) - \phi(0)) = \sigma_{\text{pn}}^2 f_s t \equiv k t \quad (\text{variance grows linearly in time}).$$

Why this sets a floor no algorithm can beat

A random walk has *independent increments*: the steps taken after your last measurement are statistically unrelated to everything you observed before it. Whatever the walk accumulates between corrections is therefore **unpredictable**, and a loop that corrects every T seconds carries at least kT of phase variance. More SNR, better estimators, smarter filters — none of it touches this term. Only a quieter oscillator or a shorter T does.

[proj] default: $\sigma_{\text{pn}} = 2 \times 10^{-4}$ rad/sample at 1 MS/s $\Rightarrow k = 0.04$ rad²/s
 $\Rightarrow \sqrt{kT} = 45$ mrad at $T = 50$ ms. Remember this number.

2.3 The other noise types

type	what it is	effect	ADEV
white PM	i.i.d. jitter per phase sample	averages away as $1/L$	τ^{-1}
white FM	random-walk phase (prev. slide)	the kT floor	$\tau^{-1/2}$
flicker FM	$1/f$ frequency noise	slow wander, no exact model	τ^0
RW FM	random walk <i>in frequency</i>	aging, temperature	$\tau^{+1/2}$

- **White PM:** the phase *reading* jitters but nothing accumulates — averaging many samples suppresses it. It matters only inside individual measurements, never for long-term tracking.
- **Flicker FM:** frequency noise with a $1/f$ spectrum — equal noise power per *decade* of timescale, so it neither averages away nor settles into a clean random walk. No exact finite-state model exists; the standard surrogate is a sum of a few first-order low-pass (AR(1)) processes with time constants spread across the timescales of interest, whose summed spectra approximate $1/f$.
- **Random-walk FM:** the frequency itself wanders (temperature, aging), so phase *accelerates* away — the reason frequency must be tracked as its own state, not inferred from phase alone.

2.4 Grading oscillators: the Allan deviation

Ordinary variance fails for clocks: a drifting oscillator's frequency has no settled mean, so “the variance of y ” just grows with how long you watch. The field's fix is the **Allan deviation** $\sigma_y(\tau)$:

- 1 Take the fractional frequency $y = \dot{\phi}/(2\pi f_c)$ (dimensionless — “parts in 10^{11} ” language).
- 2 Average it over consecutive windows of length τ .
- 3 $\sigma_y(\tau)$ = the typical difference between *successive* window averages (divided by $\sqrt{2}$).

Differencing neighboring windows cancels slow drift, so the statistic converges — and each noise type leaves its own slope vs. τ (previous slide's table), making an ADEV plot a fingerprint of the noise mix.

How to read the number used throughout this lecture

$\sigma_y(1\text{s}) = 3.5 \times 10^{-11}$ means: two successive 1-second frequency averages typically differ by 3.5 parts in 10^{11} — at 915 MHz, by 0.032 Hz. Grades: **TCXO** (temperature-compensated crystal, the few-dollar part in most radios) sits at $\sigma_y \sim 10^{-9}$ – 10^{-10} ; **OCXO** (crystal kept in a small oven) is 10–100× steadier but bigger, costlier, power-hungry. [\[proj\]](#) default sits between: realistic, not exotic.

2.5 One crystal, three synchronized errors

Inside a radio, the carrier synthesizer *and* the ADC/DAC sample clock are both multiplied up from the same reference crystal. So if the reference runs fractionally fast by y , three things happen at once:

$$\underbrace{\text{CFO} = f_c y}_{\text{carrier high}} \quad \underbrace{\text{SFO} = 10^6 y \text{ ppm}}_{\text{samples too fast}} \quad \underbrace{\text{timing drift} = y f_s T \text{ samples per interval}}_{\text{frames arrive early/late}}$$

At the [proj] defaults, an initial CFO of 1500 Hz at 915 MHz means $y = 1.64 \times 10^{-6}$, hence 1.64 ppm of SFO and 0.082 samples of frame drift per 50 ms — **one** random process seen three ways, moving together as the crystal wanders.

- A faithful simulation must derive all three from one process (many models set them independently — then they decorrelate, which real hardware cannot do).
- A digital correction (an NCO, introduced shortly) shifts the *baseband* phase/frequency; it does not touch the crystal. The physical SFO and timing drift continue regardless of corrections.

Source: Shared-reference architecture: standard SDR design; modeled in `ota_sync/sdr.py`.

3.1 Measuring a phase: the pilot and the bound

Station A transmits a *pilot*: a waveform B knows in advance. B receives L samples $r_n = e^{j\phi} s_n + w_n$ (known s_n , noise w_n) and asks: what is ϕ ? Correlating r against s and taking the angle is the maximum-likelihood answer. How good can any estimator be? The Cramér–Rao lower bound (CRLB) is the information-theoretic answer. Intuition: plot how probable the observed data is as a function of the assumed ϕ — a curve peaking near the true value. The *sharper* that peak, the more precisely the data pins ϕ down; “Fisher information” is exactly that sharpness (the curvature at the peak), and no unbiased method can have variance smaller than its inverse:

$$\text{var}(\hat{\phi}) \geq \frac{1}{2 \text{SNR} L}.$$

Each extra sample and each dB of SNR steepen the peak: twice the SNR or twice the samples $\Rightarrow$ half the variance.

At this project's settings

20 dB SNR (SNR = 100), $L = 2 \times 2047$: bound = 1.1 mrad. Phase can be measured very precisely — if noise were the only problem.

3.2 Measuring frequency: phase slope over a baseline

Frequency is the rate of phase change, so measure phase twice and divide by the time between:

$$\hat{\omega} = \frac{\hat{\phi}_2 - \hat{\phi}_1}{T_{\text{sep}}} \quad \Rightarrow \quad \text{var}(\hat{\omega}) = \frac{1}{\text{SNR} L_b T_{\text{sep}}^2}.$$

- The T_{sep}^2 in the denominator is the useful lever: a *longer baseline* between the two phase looks improves frequency quadratically. (Same reason a long telescope baseline improves angular resolution.)
- But the baseline cannot exceed the frame, and the phase difference must stay unambiguous ($|\omega| T_{\text{sep}} < \pi$) — so receivers use a *coarse* estimate from short spacing first, then a *fine* one from long spacing. We will see exactly this two-stage structure in the receiver.
- At [proj] settings ($L_b=2047$, $T_{\text{sep}}=2.175$ ms, 20 dB): $\sigma_{\hat{\omega}} = 0.16$ Hz.

Source: Two-block frequency estimation: standard; variance form as implemented in `ota_sync/sdr.py`.

3.3 The catch: your own clock jitters while you measure

The CRLB assumed the only corruption is receiver noise. But the receiver's oscillator runs its random walk *during* the pilot, and the phase estimate effectively averages that walk over the observation. For a walk observed over samples $[a, a+n]$:

$$\text{var}(\overline{W}) \approx \sigma_{\text{pn}}^2 (a + n/3)$$

(the walk arrives at the window already displaced — the a — and averaging a wandering path leaves $n/3$).

per-measurement error	phase	frequency
AWGN CRLB	1.1 mrad	0.16 Hz
own LO phase noise during pilot	8.3 mrad	0.68 Hz

- Measurement here is **phase-noise limited, not SNR limited**: more transmit power stops helping almost immediately.
- A tracking filter must be told this (its R must include the phase-noise terms), or it will trust each measurement $\sim 7\times$ too much and mis-weight everything.

Source: Walk-average variance and evaluation: [this project](#), [teaching_document.pdf](#) §3.3.

3.4 What to transmit: Zadoff–Chu pilots [proj]

A good pilot needs two properties, and Zadoff–Chu (ZC) sequences, $x[n] = e^{-j\pi un(n+1)/L}$, deliver both:

- 1 **Constant amplitude** — every sample has $|x[n]| = 1$, so the power amplifier runs at a steady level without distortion.
- 2 **Ideal autocorrelation** — correlated against a shifted copy of itself, a ZC gives zero at every nonzero (cyclic) shift. A matched filter therefore produces *one* sharp peak, giving precise timing even when multipath smears the signal. A cyclic prefix (CP; the sequence's tail copied in front) makes the channel's smearing look cyclic, preserving the property.

[proj] frame (4606 samples = 4.6 ms at 1 MS/s):

$$\underbrace{\text{ZC-}16 \times 16}_{\text{STF: 256 samples}} \rightarrow \underbrace{[\text{CP } 128 + \text{ZC-}2047] \times 2}_{\text{LTF: 4350 samples}}$$

Short repeated blocks for *finding* the packet; two long blocks for *precision*. Next slide: why that split.

Source: ZC: Chu, IEEE Trans. Inf. Theory, 1972. Note [RN23] transmits no waveform at all (its Eq. 15 observes oscillator states directly).

3.5 The receiver, stage by stage [proj]

- ❶ **Detection + coarse CFO (from the STF).** Correlate the signal against *itself* delayed by one repetition (16 samples). A repeated waveform matches itself regardless of channel or frequency offset — that is what makes detection robust. And a frequency offset rotates each repetition relative to the last by $\omega \cdot 16 T_s$, so the *angle* of that self-correlation reads off the CFO. Unambiguous while the rotation is under half a turn: $|f| < f_s/32 = 31.25 \text{ kHz}$. That is the acquisition range.
- ❷ **Fine timing.** Cross-correlate against the full known preamble; the ZC property gives one sharp peak (sample-accurate).
- ❸ **Fine CFO (from the two LTFs).** Phase difference across the 2175-sample baseline — the long-baseline trick of slide 3.2, now that coarse CFO has removed the ambiguity risk.
- ❹ **Phase.** After removing the measured CFO, matched-filter the LTFs; the correlation angle is the frame phase.

Each stage needs the one before it: detect $\rightarrow$ time $\rightarrow$ frequency $\rightarrow$ phase.

Source: Repeated-block detection/CFO: Schmidl & Cox, IEEE Trans. Commun., 1997. Implementation: `ota_sync/sdr.py`.

3.6 What the measurement actually contains

Here is the central obstacle of the whole subject. The phase B measures is

$$m = \underbrace{\phi_{\text{tx}} - \phi_{\text{rx}}}_{\theta: \text{ oscillators, what we want}} + \underbrace{\phi_c}_{\text{channel: delay + multipath}} + n.$$

The wave accumulated phase traveling from A to B. The receiver sees one number; θ and ϕ_c enter it *identically*. No amount of processing of one-way data can tell them apart.

Consequence

A one-way loop that drives $m \rightarrow 0$ does not set $\theta = 0$; it sets $\theta = -\phi_c$. The receiving oscillator *absorbs the channel phase* as a hidden bias. Measured at [proj] defaults: $\phi_c = +2.92$ rad — so one-way sync leaves the oscillators 2.92 rad apart while every internal indicator reads “locked.”

Whether that bias matters depends on the application — next slide.

Source: Identifiability framing and the measured bias: this project (`--model sdr`).

3.7 When the bias is harmless, and when it is fatal

Harmless: a user closes the loop (cell-free MIMO). If a user measures pilots from both stations, its channel estimate for station B — called CSI, *channel-state information* — contains B's oscillator bias, indistinguishable from the channel to B and corrected together with it. Any *constant* per-station phase drops out. Only the *drift* between CSI updates matters, and the sync loop controls exactly that. One-way sync suffices. **Fatal: nothing closes the loop (passive coherent sensing).** Beamforming toward a computed direction, or coherently combining echoes from a target that cannot talk back: the carriers must be aligned *at the antennas themselves*. A hidden 2.92 rad offset simply points the array somewhere else — and nothing in the system can see it.

So the question becomes

Can the stations measure θ *without* the channel phase? Yes — by using the channel twice.

Source: Application taxonomy: standard; quantified for both cases later in this deck.

3.8 Two-way exchange: canceling the channel with itself

Radio channels are *reciprocal*: the phase accumulated from A to B equals the phase from B to A over the same path at the same time. So exchange pilots both ways within a channel coherence time:

$$m_f = \theta + \phi_c + n_1 \quad m_r = -\theta + \phi_c + n_2$$

(θ flips sign with direction; ϕ_c does not). Then:

$$\frac{1}{2}(m_f - m_r) = \theta + \frac{n_1 - n_2}{2} \quad \frac{1}{2}(m_f + m_r) = \phi_c + \frac{n_1 + n_2}{2}$$

The half-difference is the pure oscillator offset; the half-sum is the pure channel phase; the two are uncorrelated, each with half the noise.

Two costs, both of which will bite later

(1) Phases are only known modulo 2π , and *halving* a modulo- 2π quantity leaves two possible answers, π apart. One branch must be chosen. (2) Only the *air* is reciprocal: each node's transmit chain and receive chain are different electronics, and that per-node TX/RX difference survives the subtraction — it needs a one-time loopback calibration.

4.1 Closing the loop: the NCO

B cannot bend its crystal. What it *can* do is rotate its digital baseband: multiply samples by $e^{-j(\hat{\phi} + \hat{\omega}t)}$ — a **numerically controlled oscillator (NCO)**. Corrections are digital, quantized (here: phase steps of $2\pi/2^{16}$, frequency steps of 0.01 Hz), and take effect only after the frame has been processed — one interval of *latency* in this model. The loop, once per interval T :

capture $\rightarrow$ estimate $(m, \hat{\omega}) \rightarrow$ filter $\rightarrow$ command NCO (loads next interval)

Two questions remain, and they are the last two foundations:

- 1 How should noisy measurements be turned into corrections? (the Kalman filter, next slide)
- 2 What does the loop's delay cost? (slide after)

Source: NCO discipline: standard SDR practice; parameters: `ota_sync/sdr.py`.

4.2 The Kalman filter, as a trust dial

Each interval the filter holds a *prediction* of $[\theta, \omega]$ and receives a *measurement*. It blends them:

$$\text{estimate} = \text{prediction} + \kappa (\text{measurement} - \text{prediction})$$

The gain $\kappa \in [0, 1]$ is not tuned by hand; it is computed from two declared variances:

- Q (process noise): *how fast does the truth wander between measurements?* — the oscillator physics of Part 2.
- R (measurement noise): *how noisy is one measurement?* — the estimation limits of Part 3, including the phase-noise terms.

Large Q /small R : trust the meter ($\kappa \rightarrow 1$). Small Q /large R : trust memory ($\kappa \rightarrow 0$). After a few steps κ and the error variance settle at a **steady state** — the equilibrium where information gained per measurement balances truth-wander per interval. That steady-state uncertainty is a computable number, and it is about to matter. (One word to keep: the filter’s belief just after absorbing a measurement is its *posterior*; “posterior uncertainty” means the spread of that belief.) Phase wraps at $\pm\pi$, so the filter observes $[\cos m, \sin m]$ instead of m (no wrap discontinuity) and re-linearizes within each update — an *extended* KF.

Source: Kalman filtering: standard; iterated-EKF and Joseph-form details in `ota_sync/core.py`.

4.2b The trust dial in action: a toy example

One tracked quantity. The truth wanders by variance $Q = 4$ per step (steps of typical size 2); each measurement has variance $R = 5$ (typical error 2.24). Track the filter's own uncertainty P :

$$\text{predict: } P^- = P + Q \quad \text{gain: } \kappa = \frac{P^-}{P^- + R} \quad \text{update: } P \leftarrow (1 - \kappa)P^-$$

round	P^- (before meas.)	κ	P (after meas.)	error std
1	4.00	0.44	2.22	1.49
2	6.22	0.55	2.77	1.66
3	6.77	0.58	2.88	1.70
∞	6.90	0.58	2.90	1.70

Three things to take away:

- The uncertainty **settles in a few rounds**, wherever it starts: the equilibrium where the wander added per step (Q) balances the information each measurement removes.
- Settled error (1.70) beats any *single* measurement (2.24) — memory helps — but cannot reach zero: fresh wander keeps arriving.
- This settled number is exactly the “steady-state posterior uncertainty” that the latency term on the next slide integrates into phase.

4.3 What latency costs

The correction computed from frame k loads at frame $k+1$ — $T = 50$ ms later. The controller is not naive: it knows its latency and commands the *predicted* state, $\hat{\theta} + \hat{\omega}T$. The deterministic drift is cancelled. What survives?

$$\theta_{\text{actual}} - \theta_{\text{commanded}} = \underbrace{(\omega - \hat{\omega})T}_{\text{frequency error, integrated}} + \underbrace{\text{new walk}}_{\text{already counted}}$$

The frequency *estimate error* integrates into phase over the wait. Its size is the filter's steady-state frequency uncertainty $\sigma_{\hat{\omega}}$ — computable from Q_{ω} and R_{ω} by iterating the variance recursion to its fixed point:

$$Q_{\omega} = 0.79, R_{\omega} = 1.03 \text{ (rad/s)}^2 \Rightarrow \sigma_{\hat{\omega}} = 0.77 \text{ rad/s} \Rightarrow \sigma_{\hat{\omega}}T = 38 \text{ mrad.}$$

The pre-registered prediction [proj]

Walk floor $\sqrt{kT} = 45$ mrad (slide 2.2) + latency floor 38 mrad + tracking ~ 12 mrad. Neither of the first two depends on SNR. Part 3 of this lecture measures exactly these numbers.

Source: Riccati fixed point: standard; this decomposition of a closed sync loop: this project (no counterpart in [RN23], whose updates are instantaneous).

5.1 The reference paper: what it is and why it exists

Full citation

M. Rashid and J. A. Nanzer, “Frequency and Phase Synchronization in Distributed Antenna Arrays Based on Consensus Averaging and Kalman Filtering,” *IEEE Transactions on Wireless Communications*, vol. 22, no. 4, pp. 2789–2802, April 2023. (arXiv:2201.08931; PDF in this repository.)

Its motivation, from its introduction [RN23, §I]:

- *Closed-loop* array methods (nodes adjust using feedback bits from the destination) only work when the destination can talk back — useless for remote sensing or arbitrary beam steering. So the paper targets *open-loop* arrays: nodes align **with each other only**.
- Prior open-loop work used a primary–secondary architecture (one master broadcasts a reference) — which dies with the master. Prior *decentralized* work aligned **frequency only**.
- This paper’s goal: decentralized **frequency-and-phase** alignment, with no leader, plus a closed-form prediction of the residual error.

5.2 [RN23] The network picture

- N nodes ($N = 5$ to 400 in their experiments), each a radio with its own oscillator. Nodes that can hear each other are joined by an edge of an undirected graph $\mathcal{G}$.
- **Connectivity** $c \in (0, 1]$: the fraction of all $N(N-1)/2$ possible edges that exist. $c = 1$: everyone hears everyone; $c = 0.05$: sparse, each node hears few neighbors. Sparse graphs are the realistic and difficult case, and much of the paper is about them.
- Once per update interval T , every node broadcasts and every node updates — *synchronized rounds*, which is why the paper assumes time synchronization is already solved.
- Simultaneous broadcasts are separated by TDMA (time slots): with D neighbors on average, each node's observation window shrinks from L samples to L/D — degrading the estimation errors below [RN23, Eqs. 18–21].

Source: [RN23] §II–III; their simulation parameters: $f_C = 1$ GHz, $f_S = 10$ MHz, $T = 0.1$ ms, SNR = 0 dB.

5.3 [RN23] What each node measures

Each node broadcasts its bare oscillator signal, and neighbors estimate its frequency and phase from L received samples:

$$x_n(t) = e^{j(2\pi f_n t + \theta_n)} + \text{noise} \quad [\text{RN23, Eq. 15}]$$

with estimation errors at the CRLBs (slide 3.1's bound, plus its frequency analog):

$$\sigma_f^m = \sqrt{\frac{6}{(2\pi)^2 L^3 \text{SNR}}} \cdot f_s \quad \sigma_\theta^m = \sqrt{\frac{2}{L \text{SNR}}} \quad [\text{RN23, Eqs. 16–17}]$$

(Why L^3 for frequency: more samples average the noise — one factor of L — and lengthen the observation window, which improves a *rate* quadratically, exactly the baseline effect of slide 3.2 — two more factors of L .)

Read Eq. 15 carefully — this is the paper's deepest assumption

The oscillator is observed *directly in noise*. There is no propagation channel between the nodes: no delay phase, no multipath. The identifiability problem of slide 3.6 — that a real receiver sees oscillator *plus* channel phase — does not exist in this model. All channel and geometry phase is bundled into one constant θ_n^e , “assumed corrected” by some other mechanism.

Part 2 measures what happens when this assumption meets a real channel

5.4 [RN23] The error model, part 1: the oscillators

Their oscillator spec, in the Allan-deviation form of slide 2.4:

$$\sigma_f = f_c \sqrt{\beta_1/T + \beta_2 T}, \quad \beta_1 = \beta_2 = 5 \times 10^{-19} \quad [\text{RN23, Eq. 2}]$$

(β_1 : white FM; β_2 : random-walk FM; values for an ordinary quartz TCXO — the few-dollar temperature-compensated crystal of slide 2.4. No flicker term.)

Evaluate it at their setup ($f_c = 1 \text{ GHz}$, $T = 10^{-4} \text{ s}$):

$$\sigma_f = 10^9 \sqrt{5 \times 10^{-15} + 5 \times 10^{-23}} \approx 70.7 \text{ Hz}$$

Two error terms follow per interval:

- **Drift phase** $2\pi\sigma_f T = 2\pi \cdot 70.7 \cdot 10^{-4} = 44.4 \text{ mrad} = 2.55^\circ$: the phase the frequency offset sweeps out before the next update.
- **Intra-interval drift** $\pi T\sigma_f = 1.27^\circ$: the frequency is not even constant within the interval; integrating its linear change leaves this [RN23, Eqs. 3–4, 13].

Source: [RN23] §II-B; numerical evaluation: this deck.

5.5 [RN23] The error model, part 2: jitter and estimation

Phase jitter. A spec-sheet number: the oscillator's total phase-noise power, $A = -53.46 \text{ dBc}$ (“dBc” = decibels relative to the carrier's own power). It converts to a jitter of $\sigma_\theta = \sqrt{2 \cdot 10^{A/10}} = 2.7 \text{ mrad} = 0.15^\circ$, *drawn fresh each interval* [RN23, Eq. 5] — no memory across intervals (contrast slide 2.2's continuous walk). **Estimation errors** at their setup ($L = Tf_s = 1000$ samples, $\text{SNR} = 0 \text{ dB} = 1$):

$$\sigma_f^m = 123 \text{ Hz} \Rightarrow 2\pi\sigma_f^m T = 4.44^\circ \qquad \sigma_\theta^m = 44.7 \text{ mrad} = 2.56^\circ$$

Now the budget is visible — and it explains the paper's agenda

Per interval: frequency-estimation $4.44^\circ >$ phase-estimation $2.56^\circ \approx$ drift $2.55^\circ >$ intra-drift $1.27^\circ \gg$ jitter 0.15° . At short T the loop is **estimation-dominated**: only 1000 noisy samples per look. That is precisely the regime where filtering (memory) should help — which is why the paper's second algorithm exists.

5.6 [RN23] DFPC, the algorithm itself

One round of *decentralized frequency and phase consensus*, per their Algorithm 1:

- ① Every oscillator degrades: frequencies pick up drift $\mathcal{N}(0, \sigma_f^2)$; phases pick up intra-interval drift and jitter.
- ② Every node broadcasts; every neighbor *estimates* what it hears, adding $\mathcal{N}(0, (\sigma_f^m)^2)$ and $\mathcal{N}(0, (\sigma_\theta^m)^2)$.
- ③ Every node replaces its own frequency and phase by a **weighted average of the estimates of its neighbors (and itself)**:

$$\mathbf{f}(k) = \mathbf{W} \mathbf{f}(k-1), \quad \boldsymbol{\theta}(k) = \mathbf{W} \boldsymbol{\theta}(k-1) \quad [\text{RN23, Eq. 8}]$$

- The matrix form is bookkeeping: row n has nonzero entries only for node n 's neighbors, so each node computes locally. No leader.
- For plain DFPC **the broadcast waveform is the message**: no data link exists anywhere.
- Note also what step 3 is *not*: there is no NCO, no latency — the new value simply *is* the node's state (instantaneous actuation, slide 4.3's term absent by construction).

5.7 [RN23] The mixing matrix, built by hand

The weights are Metropolis–Hastings: for an edge (i, j) , $w_{ij} = 1/(1 + \max\{n_i, n_j\})$ where n_i is node i 's degree; the self-weight absorbs the remainder [RN23, Eq. 9].

Worked example — three nodes in a line, $A-B-C$ (degrees 1, 2, 1):

$$w_{AB} = w_{BC} = \frac{1}{1 + \max(1, 2)} = \frac{1}{3} \quad \Rightarrow \quad \mathbf{W} = \begin{bmatrix} \frac{2}{3} & \frac{1}{3} & 0 \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ 0 & \frac{1}{3} & \frac{2}{3} \end{bmatrix}$$

Check the two properties that make consensus work:

- **Rows sum to 1:** each update is a proper average — no node can run away.
- **Columns sum to 1 too** (doubly stochastic): the *network average* is preserved by every round. So if the values converge to a common number, that number must be the average of where they started.

The max in the denominator is the safety margin: a weight small enough for the busier endpoint is safe for both.

Source: [RN23] Eq. 9; worked example: this deck.

5.8 [RN23] Why and how fast consensus converges

Think of each round as a blurring pass over the network: the overall average survives untouched, and every *pattern of disagreement* fades. Linear algebra makes that exact: $\mathbf{W}$'s largest eigenvalue is 1, belonging to the “everyone equal” direction $\mathbf{1} = (1, \dots, 1)$; every other pattern shrinks per round by at most λ_2 , the second-largest eigenvalue magnitude — the survival factor of the most stubborn disagreement pattern:

$$\mathbf{W}^l \longrightarrow \frac{1}{N} \mathbf{1} \mathbf{1}^\top \quad (\text{all states} \rightarrow \text{the initial average})$$

- Dense graph: λ_2 small, disagreement dies in a few rounds. Sparse graph: $\lambda_2 \rightarrow 1$, information creeps across the network.
- Their trajectories [RN23, Figs. 1–2]: initial frequency spread of ± 100 ppm collapses within tens of iterations at $c = 0.2$; larger N at fixed c converges *faster* (each node has $D = c(N-1)$ neighbors — more neighbors, better local averages).
- Converged total phase error falls with N toward a floor [RN23, Fig. 3] — averaging over more neighbors suppresses the per-node errors, until the injected noise floor takes over.

5.9 [RN23] Why the error does *not* go to zero

With noiseless oscillators and perfect estimates, mixing would converge exactly and stay. But *every round injects fresh errors* (steps 1–2 of the algorithm), so the network reaches an equilibrium between two competing rates:

mixing removes disagreement at $\sim (1 - \lambda_2)$ per round
vs. injection adds σ_e per round

Formally: the accumulated error after l rounds is $\sum_i \mathbf{W}^{i+1} \mathbf{e}_{l-i}$; measured relative to the network average, its standard deviation is bounded by $\sigma_e \sqrt{\sum_m \lambda_2^{2m}}$ [RN23, Eqs. 23–26].

- **Dense:** the geometric sum is small — equilibrium well below the injection level.
- **Sparse** ($\lambda_2 \rightarrow 1$): the sum is large — errors linger as long as they keep being replaced, and the equilibrium saturates *at the injected level itself*.

In the sparse limit the residual is therefore just the root-sum-of-squares of the five per-round error terms — next slide.

Source: [RN23] §III-A.

5.10 [RN23] Eq. 27, assembled from the numbers we already have

$$\sigma_{\phi, \text{total}} = \sqrt{(2\pi\sigma_f T)^2 + (2\pi\sigma_f^m T)^2 + (\pi T\sigma_f)^2 + (\sigma_\theta^m)^2 + \sigma_\theta^2} \quad [\text{RN23, Eq. 27}]$$

At their setup, the five terms are the ones computed on slides 5.4–5.5:

$$\sqrt{(2.55^\circ)^2 + (4.44^\circ)^2 + (1.27^\circ)^2 + (2.56^\circ)^2 + (0.15^\circ)^2} = 5.9^\circ$$

- Well under the $18^\circ/90\%$ bar of slide 1.2 — consensus alone suffices *in this model*.
- The biggest term is frequency estimation (4.44°): the motivation for filtering, next slide.
- Compare with slide 4.3's closed-loop floor: Eq. 27 has **no latency term** (instantaneous updates), **no channel term** (Eq. 15 has no channel), and **no noise memory** (jitter redrawn each round). These absences are exactly where Part 3 will press.

Source: [RN23] Eq. 27; term-by-term evaluation: this deck (matches the 5.9° used in Part 3's validation).

5.11 [RN23] KF-DFPC: filter first, then average

The frequency-estimation term dominates because each look uses only 1000 noisy samples — but the *truth changes slowly* between looks. That is exactly when the trust dial of slide 4.2 says: use memory. So each node runs a two-state Kalman filter on its own noisy observations before the network averages:

- State $[f_n, \theta_n]$; process covariance $\mathbf{Q}$ built from the drift and jitter statistics of slides 5.4–5.5 (its off-diagonal term encodes that a frequency kick also moves the phase it integrates into) [RN23, Eq. 29]
- Measurement covariance = the CRLBs [RN23, Eq. 31]; standard predict/update [RN23, Eqs. 33, 35–36]
- Then consensus-average the *posterior means*, and mix the covariance entries with squared weights so each node's filter knows how uncertain its mixed prior is [RN23, Eqs. 38–39]

Two structural facts

(1) Posterior means *and covariances* must now travel between nodes: KF-DFPC needs an error-free **data side channel**, which plain DFPC did not. (2) **No closed form** is derived for KF-DFPC's residual — it is characterized by simulation against Eq. 27.

5.12 [RN23] Results, part 1: convergence and baselines

Convergence speed (threshold: total phase error under 1°):

	DFPC	KF-DFPC
$N=100$, connectivity 0.05	16 iterations	2
$N=65$, connectivity 0.05	158 iterations	4

Filtering shortens convergence dramatically on sparse graphs — smoothed values mix far more efficiently than noisy ones [RN23, Figs. 4, 8]. Fewer rounds also means less energy spent broadcasting. **Baselines beaten** [RN23, Figs. 7–9] — three existing distributed estimators:

- **DLMS** (diffusion least-mean-squares): adaptive averaging with no state model — worst everywhere (it cannot exploit the slow dynamics).
- **DKF** (diffusion KF): shares estimates but not covariances — slow to converge.
- **KCIF** (Kalman consensus information filter): shares measurements and covariances — competitive at small N /high SNR, but plateaus (suboptimal gain).

KF-DFPC wins at low SNR and sparse connectivity — the practical corner.

Source: [RN23] §V, Figs. 4, 7–9.

5.13 [RN23] Results, part 2: the update-interval sweep

Their Fig. 10 sweeps T from 0.1 ms to 3 s and finds a **U-shape** for DFPC, minimum near $T = 20$ ms. The shape follows from the term scalings we computed:

- **Drift terms grow with T :** more time between updates means more phase swept out ($2\pi\sigma_f T$).
- **Estimation terms shrink with T :** a longer interval means more samples per look ($L = Tf_s$), so $\sigma_f^m, \sigma_\theta^m$ fall.
- The optimum sits where the two cross — at their parameters, $T \approx 20$ ms.

KF-DFPC removes most of the left (estimation-dominated) branch: filtering substitutes memory for samples, so short- T operation stops being punished. Consistently, its converged error is nearly **SNR-independent** across -30 to $+120$ dB [RN23, Fig. 11] — the filter compensates measurement quality with memory, at the cost of a few more convergence iterations at low SNR [RN23, Fig. 9]. Note the design language: this figure is a *cadence trade-off* — the same axis this project's designs will push on, but without latency, channel, or airtime in the model.

Source: [RN23] §V, Figs. 9–11.

5.14 [RN23] The assumptions, and what each one hides

- 1 **No channel in the measurement** (Eq. 15). Hides the identifiability problem (slide 3.6) entirely — over the air, every measurement would contain ϕ_c .
- 2 **Instantaneous updates.** Hides the latency term (slide 4.3) — in the paper, a computed correction simply *is* the new state.
- 3 **Jitter redrawn each interval; no flicker.** Hides noise memory: real intra-frame noise carries into the next interval.
- 4 **Statistics-level validation.** Its simulations draw Gaussian errors from the assumed model (Algorithm 1) — they exercise the algebra, not the physics.
- 5 **Time sync assumed done; side channel ideal.**

None of these are algebra errors — they are scope choices, and standard ones. Part 3's simulator exists precisely to measure what they cost.

Source: Assessment: this project ([paper_review_rashid_nanzer.pdf](#)), confirmed against the full text.

[proj] How the project is built: tools, and why each one

Python 3.11 + PyTorch All signal processing and simulation is tensor math over complex numbers; PyTorch supplies that, plus seeded random-number generators and an optional GPU path. Everything runs in **double precision** (float64/complex128) — the questions here live at the milliradian level, and single precision ($\sim 10^{-7}$ relative error) would leave too little headroom under long accumulations.

NVIDIA Sionna 2 (sionna-no-rt) Supplies the channel physics: the 3GPP TR 38.901 TDL multipath profiles, Jakes-correlated tap evolution under mobility, and the blocks that apply them to waveforms. Reason: the channel is the one part of the physics for which a standards-grade, widely-validated implementation exists — hand-rolling it would add exactly the kind of silent error this project exists to avoid.

Determinism by construction Every run is seeded (PyTorch global seed, a separate stream for the radio's noise draws, Sionna's own seed): identical command $\Rightarrow$ byte-identical output — which is what made the “rerun and diff” verification step meaningful.

Source: requirements.txt: sionna-no-rt==2.0.1; pyproject.toml: Python $\geq$ 3.11.

[proj] How the code is organized

module	contents
ota_sync/core.py	oscillator model, iterated EKF (shared by every loop)
ota_sync/sdr.py	config, frame, radio link + impairments, receiver, one-way loop, oracle grading path
ota_sync/coherent.py	two-way loop, π -calibration, CSI-gain evaluation
ota_sync/dfpc.py	[RN23] reimplementations: statistics level <i>and</i> OTA
ota_sync/microsync.py	two-tier micro-pilot loop
hybrid_calibration/	the 3-state hybrid (a separate package)
simulation.py	one CLI; <code>--model</code> selects the loop (7 models + compare)
tests/	18 pytest regression tests

- One design rule: every loop reuses the *same* oscillator, channel, and receiver blocks — so when two schemes differ, the difference is the scheme, not the plumbing. (This is what made the 82.8-vs-80.8 equivalence result meaningful.)
- Each finding in this lecture is frozen as a test: oracle consistency, the ablation floors, anti-phase capture, the calibration flip, the Doppler-prior effect — so later changes cannot silently undo them.

[proj] Working method: how results were kept trustworthy

The recurring pattern, applied to every claim in Part 3:

- 1 **Predict first.** Hand-compute the expected number from the physics before running (the 45 mrad walk and 38 mrad latency floors of slide 4.3 predate every simulation that confirmed them).
- 2 **Isolate by ablation.** Every physical effect has a switch (`--no-rf-impairments`, `--correction-latency 0`, `--micro-pilots M`, `--anchor-every K`) so each term of a budget can be measured alone.
- 3 **Grade against independent truth** (the oracle path) — never against the estimator's own inputs.
- 4 **Verify surprises independently.** The anti-phase capture was confirmed by hand arithmetic, by a four-line radio-free model, and by a seed sweep against the predicted capture condition — three routes that share no code with the simulator's loop.
- 5 **Check reproducibility literally:** rerun and diff (deterministic seeding makes outputs byte-identical).
- 6 **Cross-check budgets:** e.g., unfiltered consensus's measured 0.442 Hz frequency noise predicts a 146 mrad residual; observed 153.

Source: The same discipline explains the one thing simulation cannot supply: hardware validation (open item).

[proj] The simulator: test the physics, not the algebra

Everything from Part 1 is implemented at the waveform level and nothing is handed to the receiver:

carrier / rate	915 MHz / 1 MS/s	interval T	50 ms
channel	3GPP TDL-D + shadowing	noise floor	fixed (so SNR fades)
oscillators	all four noise types, continuous	clock	one crystal (slide 2.)
TX/RX chain	PA, DAC, IQ imbal., AGC, DC, ADC	NCO	quantized, latency 1

- “TDL-D”: a standardized statistical multipath profile published by the cellular-standards body 3GPP (report TR 38.901), implemented via NVIDIA’s Sionna library; “shadowing”: slow strength fades from obstructions. The other rows are the standard analog imperfections of any real transceiver.
- The receiver runs the full stage chain of slide 3.5 on raw IQ — detection can fail, timing can slip, estimates carry their real noise.
- The intra-frame phase-noise walk *continues* into the next interval (folded into the oscillator state): noise has memory, unlike [RN23].

Source: `ota_sync/` package; architecture: `simulation_design.pdf`.

[proj] Grading honestly: the oracle path

A trap this project fell into, then fixed

Compare the filter's estimate to the measurement that just updated it, and the residual is only $(1 - \kappa) \times$ the innovation (slide 4.2: the estimate was *pulled toward* that measurement). Both can be far from the truth while agreeing with each other. This code initially reported 0.105 mrad tracking error that way.

Fix: every capture is generated *twice*. The **oracle twin** shares everything deterministic — same frame, same channel realization, same timing, same PA/DAC distortion — and omits everything stochastic (noise, phase noise, receiver impairments). Running the same estimator on the twin yields the true observable phase; all errors are graded against that. Ground-truth tracking RMSE: **12.0–12.5 mrad** — $\sim 100\times$ the self-graded figure. (Method contrast: [RN23] validates against draws from its own assumptions; here the claims are checked against an independent physical model.)

Source: `ota_sync/sdr.py` (oracle path); `--model sdr, seed 0`.

[proj] The error budget, measured by ablation

Slide 4.3 predicted the floor before any simulation. Now measure it, isolating each term by switching physics off:

configuration	tracking	residual	what the row isolates
full defaults	12.0 mrad	69.6 mrad	all terms together
RF impairments off	1.1 mrad	34.3 mrad	latency alone (predicted 38)
zero latency	12.3 mrad	12.3 mrad	tracking alone

Reading the table: with impairments off, the walk and phase-noise terms vanish and the 34.3 mrad left is the latency term — against slide 4.3’s hand-computed prediction of 38. The 1.1 mrad tracking in that row sits exactly on the AWGN CRLB (slide 3.1). Full defaults combine $45 \oplus 38 \oplus 12$ plus flicker/shadowing $\rightarrow$ 69.6.

Consequences

The floor is set by **oscillator quality, correction cadence, and latency** — not pilot energy. And the latency term is absent from [RN23, Eq. 27] and (per a 19-source verified search) from the published literature through Dec. 2025.

Source: `--model sdr with --no-rf-impairments / --correction-latency 0`; search: LITERATURE_REVIEW.md.

[proj] Two-way sync: the bias is gone

Implementing the exchange of slide 3.8 in the simulator: forward and reverse frames traverse the same channel realization within the interval, and the EKF consumes the half-difference phase and frequency (each with half the one-way noise).

	one-way loop	two-way loop
oscillator residual	−2.92 rad (hidden bias)	81.8 mrad RMS
two-station gain	near 0 open-loop	99.83%
airtime	9.6%	19.1%

The carriers are now genuinely aligned *at the antennas* — the capability passive sensing requires — at double the pilot airtime.

But slide 3.8's caveat (1) is still owed: halving a wrapped quantity leaves two candidate answers, π apart, and a branch must be picked. The next slide traces what a wrong pick does.

Source: `ota_sync/coherent.py`, `--model twoway`, seed 0.

[proj] A wrong branch is a stable illusion

The loop resolves the two candidates by picking the one nearest its current belief. Trace a wrong pick at acquisition, step by step:

- ➊ **Acquisition.** True offset $\theta = 1.94$ rad; candidates $\{1.94, -1.20\}$. With no prior belief the rule “nearest zero” picks -1.20 — wrong by exactly π . The filter now believes $\hat{\theta} = \theta - \pi$.
- ➋ **Correction.** The loop drives the *believed* phase to zero. Since belief is π off, the *true* relative phase parks at π : perfect anti-phase.
- ➌ **Every later measurement confirms the belief.** The half-difference again yields two candidates, $\{\approx 0, \approx \pi\}$; the one nearest the (near-zero) belief is chosen — always the shifted one. Innovations are tiny, covariance settles, lock indicators glow green.
- ➍ **Nothing internal can reveal it.** The measurement simply cannot distinguish θ from $\theta - \pi$ — that information does not exist inside the loop. Only an *external* observation (does the pair combine constructively?) carries the missing bit.

This is why the resolution (next slide) must be a one-time *combining* check, not more filtering.

[proj] The π ambiguity: how it hid, how it is resolved

Why every early run was (accidentally) fine

Phase advance per interval = $2\pi \times 1500 \text{ Hz} \times 50 \text{ ms} = 75.000$ carrier cycles — an *exact integer*. So the initial 1.2 rad offset reappeared unchanged at every acquisition, and since $1.2 < \pi/2$, “pick the candidate nearest zero” happened to be right every time. A later configuration used 5 ms steps: 7.5 cycles $\Rightarrow$ the acquisition phase landed at 1.94 rad ($> \pi/2$) $\Rightarrow$ wrong branch $\Rightarrow$ stable lock at π , 0.01% gain.

Resolution (as a deployment would): after lock, one coarse combining check — constructive or destructive is *one bit* of external information — and flip the NCO by π if destructive. One further subtlety, learned the hard way: **do not reset the filter at the flip**. The half-difference measurement cannot tell π apart anyway (that is the whole ambiguity), so the filter’s near-zero state is still valid — it simply re-attaches to the true branch. Resetting it by π re-creates the shifted lock.

Source: Incident, diagnosis, fix: `teaching_document.pdf §11; --model micro --micro-pilots 9` reproduces it.

[proj] Reimplementing [RN23]: does it reproduce?

Level 1 — statistics level (their model, faithfully: Algorithms 1–2, MH weights, covariance mixing), at their setup ($N=20$, connectivity 0.2, $T=0.1$ ms, 0 dB):

	residual	[RN23, Eq. 27] bound
DFPC	8.0°	5.9° (sparse-limit bound)
KF-DFPC	2.1°	—

Consensus converges to the average; the residual sits at the bound's scale; filtering reduces it $\sim 4\times$. **Both of the paper's claims reproduce in its own model.** **Level 2 — physical level** ($N=2$, full simulator): with two nodes the MH weight is $w_{12} = \frac{1}{2}$, so “consensus” means *each node retunes its NCO by half its measured offset toward the other*, both simultaneously — now with real channels, latency, and quantization. What happens is the next slide.

Source: `ota_sync/dfpc.py`; tests in `tests/test_ota_sync.py`.

[proj] Anti-phase capture, step by step

Naive OTA consensus: each node averages in its *raw one-way* measurement — which contains ϕ_c (slide 3.6). In the relative coordinate $r = \theta_A - \theta_B$, one round is

$$r' = r + \frac{1}{2} \text{wrap}(-r + \phi_c) - \frac{1}{2} \text{wrap}(r + \phi_c).$$

Walk through the default realization, $\phi_c = 2.922$, $r_0 = 1.2$:

$$\text{wrap}(1.2 + 2.92) = \text{wrap}(4.12) = -2.16 \quad \text{wrap}(-1.2 + 2.92) = +1.72$$

$$r_1 = 1.2 + \frac{1.72}{2} - \frac{-2.16}{2} = 1.2 + 0.86 + 1.08 = 3.14 = \pi.$$

One argument wrapped ($4.12 > \pi$), the other did not — their disagreement is exactly 2π , and half of it lands the pair at π . And π is *stable*: there, both wraps give the same value ($-\pi \equiv \pi$), so $r' = \pi$ forever.

Verified three independent ways

full simulation ($r_1 = -3.117$; deterministic across reruns) · a four-line iteration of the equation, no radio code · a seed sweep matching the capture condition $|r_0| + |\phi_c| > \pi$ exactly

No wrap $\Rightarrow$ the halves cancel ϕ_c and the pair aligns: the failure is *realization-dependent*, and invisible in [RN23]'s channel-free model.

[proj] Head-to-head under identical physics

approach	origin	residual	gain	airtime
two-tier micro-pilot	[proj]	28.1 mrad	99.98%	26.0%
hybrid 1-way + anchors	[proj]	33.8 mrad	99.97%	14.9%
two-way EKF	[proj]	80.8 mrad	99.84%	19.1%
KF-DFPC + reciprocity	[RN23] alg. + our PHY	82.8 mrad	99.83%	19.1%
DFPC + reciprocity	[RN23] alg. + our PHY	153.0 mrad	99.42%	19.1%
DFPC naive	[RN23] as published	3009 mrad	0.68%	19.1%

- “+ reciprocity” = their update rule fed the channel-free half-difference (using the side channel [RN23] already assumes).
- **Unfiltered consensus is $2\times$ worse** because raw frequency noise (0.442 Hz/measurement) feeds the NCOs directly and drifts
 $2\pi \cdot 0.442 \cdot 0.05 = 139$ mrad per interval; $139 \oplus 45 = 146$ predicted, 153 observed.
- **Filtered consensus $\approx$ our EKF** (82.8 vs. 80.8): two different well-designed filtered loops reach the *same* physical floor — the algorithm layer is interchangeable; the physics of slide 4.3 binds.

[proj] Design 1: two-tier micro-pilots — the reasoning

Chain of reasoning from the budget (slide on ablation):

- 1 Both dominant terms — walk kT and latency $\sigma_{\hat{\omega}} T$ — scale with the correction period T . So correct more often.
- 2 But the full 4.6 ms frame exists for *acquisition*: detection, timing, wide-range CFO. Once locked, timing drifts 0.016 samples per 10 ms and frequency is known to ~ 0.2 Hz — none of that machinery is needed again.
- 3 Therefore: keep the full two-way frame once per 50 ms, and between frames send a *bare* known sequence (ZC-255, 0.29 ms) — correlate it at the predicted arrival, after derotating by the current frequency estimate. Phase measured; corrections issued every sub-interval.

micro-pilots M per interval	0	2	4	9
residual (mrad)	71.5	35.3	27.9	22.8
airtime	19.1%	22.6%	26.0%	34.6%

Scaling check: budget with $T \rightarrow T/(M+1)$ predicts 23–25 mrad at $M=4$; measured 27.9. $3\times$ tighter for $+7\%$ airtime.

Source: `ota_sync/microsync.py, --model micro`. No counterpart in [RN23] (one pilot class; no airtime dimension).

[proj] Design 2: hybrid calibration — the reasoning

Look again at slide 3.8: the *only* thing the return direction buys is separating θ from ϕ_c . But the two behave completely differently in time:

- θ wanders at the oscillator rate — fast, power-law noise
- ϕ_c between fixed stations is nearly constant

Paying double airtime every interval to re-measure a constant is waste. So: **estimate ϕ_c as a state of its own**, with a 3-state EKF over $[\theta, \omega, \phi_c]$:

- **One-way pilots** (cheap, every sub-interval) observe the *sum* $\theta + \phi_c$. Alone, they cannot split it — the direction $\theta - \phi_c$ is unobservable — but the filter's priors can: fast innovations get attributed to θ (large q_θ), slow ones to ϕ_c (small q_c).
- **Two-way anchors** (expensive, every K intervals) observe θ and ϕ_c *separately* (half-difference / half-sum) and re-pin the split before prior-based attribution drifts.

The design question “how often must we pay for reciprocity?” becomes “how fast does ϕ_c actually change?” — a *channel* question, not an oscillator question.

Source: `hybrid_calibration/ package, --model hybrid --anchor-every K`.

[proj] How can priors split a sum the pilots cannot separate?

This deserves care, because it is the load-bearing idea of the hybrid — and its failure mode.

- A one-way pilot observes only $\theta + \phi_c$: its sensitivity to the two states is *identical* (Jacobian row $[1, 0, 1]$). From one-way data alone, the direction $\theta - \phi_c$ is invisible.
- So when a measurement disagrees with the prediction (an *innovation*), the filter cannot know which state moved. It distributes the blame **in proportion to each state's prior uncertainty**: the Kalman gain is built from PH^T , so θ 's share $\approx P_{\theta\theta}$ and ϕ_c 's share $\approx P_{cc}$.
- Between anchors, $P_{\theta\theta}$ grows fast (oscillator physics, large q_θ) while P_{cc} grows slowly (channel believed static, small q_c) — fast innovations get charged to the clock, slow trends to the channel. **The prior is the separator.**
- This predicts the failure mode before running it: if the channel actually moves but q_c says it cannot, the channel's share of blame is ≈ 0 ; the clock absorbs everything, and the NCO chases the channel. (The Doppler experiment two slides ahead shows exactly this.)
- Anchors measure θ and ϕ_c *separately*, collapsing both variances — resetting the proportions before prior-based attribution drifts too far.

[proj] Hybrid validation 1: static channel

Prediction: with ϕ_c constant, anchors should barely matter.

anchor spacing K	1	5	20
residual (mrad)	31.8	32.9	34.0
airtime	22.6%	14.9%	13.5%

(fully two-way control: 27.9 mrad at 26.0% airtime)

A $20\times$ change in reciprocity cadence moves the residual by 2 mrad, while sync airtime falls to 13.5% — below even the plain two-way loop's 19.1%. On a static link, reciprocity can be paid for at 1 Hz or slower. **Anchor cadence has decoupled from oscillator quality.**

Source: Sweep in `hybrid_calibration/README.md` (60 intervals, seed 0).

[proj] Hybrid validation 2: moving channel (the converse)

Motion changes the path length continuously, which shifts the received frequency — the *Doppler* effect, $f_D = v/\lambda = 3.05 \text{ Hz per m/s}$ at 915 MHz. A frequency shift on the channel means its phase ϕ_c now *drifts*, up to $2\pi f_D T$ per interval — so anchors and priors *should* start to matter:

m/s	scheme	residual	gain
0.2	$K=5$, static prior	446 mrad	95.3%
0.2	$K=5$, matched prior	235 mrad	98.7%
0.2	$K=20$, static prior	1752 mrad	55.1%
0.5	$K=1$, static prior	128 mrad	99.6%
0.5	$K=1$, matched prior	44 mrad	99.95%

(fully two-way controls: 43 / 33 mrad — immune, since their measurements are channel-free)

Why the static prior fails: the filter must decide whether a drifting *sum* means the clock moved or the channel moved. Told the channel cannot move, it blames the clock — and steers the NCO to chase the channel. The matched prior ($\sigma_c \approx 2\pi f_D T$) fixes the attribution; sparse anchors are only partly rescued, because priors re-attribute but only anchors truly *separate*.

Source: Sweep in hybrid_calibration/README.md (60 intervals, seed 0).

[proj] Why user CSI absorbs static biases: two lines of algebra

The claim from slide 3.7, now proved. At CSI time t_0 the user measures each station's *effective* channel — true channel times the station's carrier phase at that moment:

$$g_i = h_i e^{j\phi_i(t_0)}.$$

For joint transmission each station then weights its signal by g_i^* (transmit “against” the measured channel). What the user receives later, at time t :

$$\sum_i g_i^* h_i e^{j\phi_i(t)} = \sum_i |h_i|^2 e^{j[\phi_i(t) - \phi_i(t_0)]}.$$

Everything static — the channel phase, the one-way sync bias, chain phases — appears in both $\phi_i(t)$ and $\phi_i(t_0)$ and cancels *exactly*. What misaligns the sum is only the **drift since the last CSI measurement**, and only its *difference* between stations (a common drift rotates everything together).

Measured with the one-way loop's residual drift as $\phi_i(t)$: mean JT gain 100% at 50 ms CSI staleness, degrading only to 99.56% at 1 s. One-way sync + ordinary CSI is fully sufficient for this application class.

Source: Evaluation: `--csi-gain (ota_sync/coherent.py)`.

[proj] Which loop for which application

Recall slide 3.7's split; now with numbers:

CSI-aided (user feedback)

- static biases absorbed by user CSI
⇒ one-way suffices
- what matters: differential drift between CSI updates
- measured JT gain vs. CSI staleness: 100% @ 50 ms → 99.56% @ 1 s

Open-loop (passive sensing)

- carriers must align at the antennas
⇒ two-way or hybrid required
- bounded closed-loop residual ⇒ fixed 0.1–0.7% gain cost at *any* integration time
- dwell limited by target/channel dynamics, not by the stations' clocks

The second column is the property passive coherent sensing needs: an *uncorrected* pair of oscillators decorrelates and caps the dwell; a closed loop holds the error bounded forever.

Source: JT evaluation: `--csi-gain (ota_sync/coherent.py)`.

Summary

- ① **Grading:** compare estimators to independent ground truth (oracle path), never to their own inputs — a $100\times$ difference here [proj]
- ② **The floor:** walk $\sqrt{kT} + \text{latency } \sigma_{\omega} \ell T + \text{tracking}$; hand-predicted from physics, then verified by ablation. The latency term is absent from [RN23, Eq. 27] and unpublished through Dec. 2025 [proj]
- ③ **The paper:** reproduces faithfully in its own model; over a real channel its naive form is bistable (anti-phase capture — walked through and triple-verified), and its filtered form, given reciprocity, meets our EKF at the same physical floor [proj]
- ④ **Moving the floor:** cadence, not energy — micro-pilots (22.8–28 mrad), and hybrid calibration (33 mrad at 13–15% airtime) whose reciprocity cadence is set by channel coherence, validated under Doppler in both directions [proj]

Open: channel-Doppler 4th state; $N > 2$ wrap-safe consensus; delayed-PLL literature check; **hardware validation — every number in this deck is simulation.**

References

The reference paper

- [RN23] M. Rashid, J. A. Nanzer, “Frequency and Phase Synchronization in Distributed Antenna Arrays Based on Consensus Averaging and Kalman Filtering,” *IEEE Trans. Wireless Commun.* 22(4):2789–2802, 2023. DOI 10.1109/TWC.2022.3213788.

Classical results used in Part 1

- D. Chu, “Polyphase codes with good periodic correlation properties,” *IEEE Trans. Inf. Theory*, 1972.
- T. Schmidl, D. Cox, “Robust frequency and timing synchronization for OFDM,” *IEEE Trans. Commun.*, 1997.
- D. Rife, R. Boorstyn, “Single-tone parameter estimation from discrete-time observations,” *IEEE Trans. Inf. Theory*, 1974.
- IEEE Std 1139 (frequency-stability metrics); J. A. Nanzer et al., *IEEE TMTT* 65(5), 2017 (the 18°/90% threshold).

This project (all in this repository)

- `teaching_document.pdf` (full derivations, same order as this deck); `simulation_design.pdf`; `paper_review_rashid_nanzer.pdf`; `LITERATURE_REVIEW.md` (verified 19-source prior-art pass); code: `ota_sync/`, `hybrid_calibration/`; entry point `simulation.py`; 18 tests.